

Object oriented programming.

course objectives

- To understand basic concepts of oop
- To implement real life objects in programming
- Reuse a common logic to reduce development time & ensure a higher level of accuracy
- To understand the principle of inheritance
- Solve broad based engineering problems by applying basic knowledge of oop language

```
#include <stdio.h>
```

```
void main ()
```

```
{
```

```
int a, b, c;
```

```
printf("Enter val of a & b");
```

```
scanf("%d %d", &a, &b);
```

```
c = a + b;
```

```
printf("Addition = %d", c);
```

```
#include <iostream>
```

```
void main ()
```

```
{
```

```
int a, b, c;
```

```
cout << "Enter val a
```

```
& b";
```

```
cin >> a >> b;
```

```
c = a + b;
```

```
cout << "Addition = "
```

```
<< c << endl;
```

```
}
```


Q2 Write a C++ program to perform arithmetic operations using Switch case.

```

→ #include <iostream>
using namespace std;
void main()
{
    double num1, num2;
    char op;

    cout << "Enter two numbers";
    cin >> num1 >> num2;

    cout << "Enter an operator (+, -, *, /, %.)";
    cin << op;

    switch (op) {
        case '+':
            cout << "Result:" << (num1 + num2)
            << endl;
            break;

        case '-':
            cout << "Result:" << (num1 - num2)
            << endl;
            break;

        case '*':
            cout << "Result:" << (num1 * num2)
            << endl;
            break;
    }
}

```

```
case '/':
```

```
if (num2 != 0)
```

```
{ cout << "Result: " << (num1/num2) << endl;
```

```
else
```

```
{ cout << "Error: " << endl;
```

```
break;
```

```
case '%':
```

```
if (num2 != 0)
```

```
{ cout << "Result: " << (num1%num2) << endl;
```

```
else
```

```
{ cout << "Error: " << endl;
```

```
break;
```

```
default:
```

```
{ cout << "Invalid operator" << endl;
```

```
}
```

```
}
```

```
return 0;
```

Q3 program to check if no. is even or odd.

```
-> #include <iostream.h>
```

```
using namespace std;
```

```
void main () {
```

```
int num;
```

```
cout << "Enter a number";
```

```
[in >> num;
```



```

if ( num % 2 == 0 )
    cout << num << " is even" << endl;
else
    cout << num << " is odd" << endl;
}
return 0;

```

4) C++ code to print 1-10 number using loop

```

→ #include <iostream.h>
using namespace std;
int main()
{

```

```

    int i;
    for (i = 1; i <= 10; i++)
    {

```

```

        cout << i << " ";
    }

```

```

    cout << endl;
}

```

```

    return 0;
}

```


5 C++ code to print numbers 10-1 using while loop.

```
> #include <iostream.h>
using namespace std;
void main ()
{
    int i = 10;

    while (i >= 1)
    {
        cout << i << " ";
        i--;
    }
    cout << endl;
}
return 0;
```

6 C++ code to print following pattern

i)

```

      *
     * *
    * * *
   * * * *
  
```

```
> #include <iostream.h>
using namespace std;
void main () {
    int i, j;
    for (i = 1; i <= 5; i++) {
```

```
for (j=1; j <= 5+i; j++) {
```

```
    cout << " ";
```

```
}
```

```
for (j=1; j <= i; j++)
```

```
    cout << "*" ;
```

```
}
```

```
    cout << endl;
```

```
}
```

```
return 0;
```

ii)

1

2 2

3 3

4 4 4 4

5 5 5 5 5

```
#include <iostream.h>
```

```
using namespace std;
```

```
void main()
```

```
{
```

```
    int i, j;
```

```
    for (i=1; i <= 5; i++) {
```

```
        for (j=1; j <= i; j++) {
```



```

cout << i << " ";
}
cout << endl;
}
return 0;

```

iii)

```

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

→

```

#include <iostream.h>
using namespace std;
void main () {
    int i, j;

```

```

    for (i = 1; i <= 5; i++) {

```

```

        for (j = 1; j <= i; j++) {

```

```

            cout << j << " ";

```

```

        }

```

```

        cout << endl;

```

```

    }
}

```

```

    return 0;
}

```

8/17/25

Experiment - 1

- 1 Write a program to declare class 'student' having data members as roll_no and name and class.

```
-> #include <iostream>
#include <string>
using namespace std;
```

```
class student
{
```

```
    int rollno;
    string name;
```

```
public:
```

```
    void accept()
```

```
{
```

```
    cout << "enter rollno and name: ";
```

```
    cin >> rollno >> name;
```

```
}
```

```
    void display()
```

```
{
```

```
    cout << "name: " << name << endl;
```

```
    cout << "rollno: " << rollno << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
    student s1;
```

```
    s1.accept();
```

```
    s1.display();
```

```
    return 0;
```

```
}
```


output:-

enter roll and name:- 22 kartik
Name:- kartik
rollno:- 22

Q2 Write a program to declare class 'Book' having data members as book_name, price and no. of pages. Accept ~~for~~ data for 2 objects.

```
→ #include <iostream>
#include <string>
using namespace std;
```

```
class book
{
```

```
public:
```

```
    string b_name;
```

```
    int b_pages;
```

```
    int b_price;
```

```
    void accept();
```

```
    {
```

```
        cout << "enter book name, book price and book pages: ";
```

```
        cin >> b_name >> b_price >> b_pages;
```

```
    }
```

```
    void display();
```

```
    {
```

```
        cout << "book name: " << b_name << endl;
```

```
        cout << "book price: " << b_price << endl;
```

```
        cout << "book pages: " << b_pages << endl;
```

```
    }
```

```
};
```

```
    book b1, b2;
```

```
    b1.accept();
```

```
    b1.display();
```



```
b2.accept();  
b2.display();
```

```
return 0;  
}
```

output:-

enter book name, book price and book pages
Science 145 200

book name:- science

book price:- 145

book pages:- 200

enter book name, book price and
book pages:- maths 150 200

book name:- maths

book price:- 150

book pages:- 200

C) Write a program to declare class "time", accept time in HH:MM:SS format, convert it into total seconds.

→ #include <iostream>
using namespace std;

class timesec

{

int h, m, s;

char c;

public:

void accept()

{

cout << "enter time in HH:MM:SS
format: ";

cin >> h >> c >> m >> c >> s;

}

void display()

{
int total seconds = h*3600 + m*60 + s;

cout << "Time in seconds:" << total
seconds << endl;

}

};

int main()

{
timesec t;

t.accept();

t.display();

return 0;

}

enter time in HH:MM:SS format :- 283:22:23
Time in seconds :- 84143

15/7/25

Experiment - 2

Q. WAP to declare a class 'city' having data member as name and population. Accept This data for 5 cities having highest population.

```

-> #include <iostream>
using namespace std;
class city
{ public:
    String name;
    int population;
public:
    void accept()
    {
        cout << "enter the name of city: " << endl;
        cin >> name;
        cout << "enter population: " << endl;
        cin >> population;
    }
    void display()
    {
        cout << "Name: " << name << endl;
        cout << "population: " << population << endl;
    }
};
city [5];
int main()
{
    int i;
    for(i=0; i<5; i++)
    {

```


([i]. accept)

int max=0;

for(i=0; i<5; i++)

{ if ([i]. population > [max]. population)

{ max=i;

} cout << "The city with maximum population:" <<

endl;

[max]. display();

return 0;

Q WAP to declare a class 'Account' having data members as Account no. and balance. Accept this data for 10 accounts and give interest of 10% where balance is equal or greater than 5000 and display.

```

-> #include <iostream>
using namespace std;
class account
{
    int acc no;
    int balance;
public:
    void accept ()
    {
        cout << "Enter account number:" << endl;
        cin >> acc no;
        cout << "Enter balance:" << endl;
        cin >> balance;
    }
    void display ()
    {
        cout << "Account number: " << acc no << "
        Balance: " << balance << endl;
    }
    void add interest ()
    {
        if (balance >= 5000)
        {
            int interest = balance * 0.10;
            balance += interest;
            cout << "Interest of 10% added, " <<
            balance << endl;
        }
    }
}

```



```

    }
    else
    {
        cout << "Balance is less than 5000, no
        interest is added." << endl;
    }
}

```

```

int get Balance ()
{
    return Balance;
}
;

```

```

int main ()
{

```

```

    account accounts [10];

```

```

    int num Accounts;

```

```

    cout << "Enter number of accounts (max 10)
    (in >) num Accounts;

```

```

    for (int i = 0; i < num Accounts; i++)
    {

```

```

        cout << "In Account" << (i+1) << ":" << endl;
        accounts [i]. accept();
    }

```

```

    cout << "Account display with 10%
    interest" << endl;

```

```

    for (int i = 0; i < num Accounts; i++)
    {

```

```

        cout << "Account" << (i+1) << ":" << endl;
        accounts [i]. display();
    }
}

```

```

accounts[i].add interest ();
}
return 0;
}

```

- Q WAP to declare a class 'staff' having data members of book title, author, name & price. Accept & display the info for 1 object using pointer of that object.

```

→ #include <iostream>
using namespace std;
class "staff"
{
    String name;
    string post;
public:
    void accept ()
    {
        cout << "Enter name: ";
        cin >> name;
        cout << "Enter post: ";
        cin >> post;
    }
    void display ()
    {
        cout << "Name: " << name << ", post: "
        << post << endl;
    }
    string get post ()
    {
        return post;
    }
}

```



```
string get name()
{
    return name;
}
```

```
int main ()
{
    staff s[5];
    bool found HOD = false;
```

```
for (int i = 0; i < 5; i++)
{
    cout << "staff member" << (i+1) << " : " << s[i].accept() << endl;
}
```

```
for (int i = 0; i < 5; i++)
{
    if (s[i].get post() == "HOD")
    {
        cout << "HOD is: " << s[i].accept() << endl;
        Found HOD = True;
        break;
    }
}
```

```
if (Found HOD)
    cout << "HOD not found." << endl;
return 0;
}
```

30/7/25

Exp - 3

- D) WAP to declare a class 'book' containing data members as book - title, author - name and price. Accept and display the info for an obj using pointer.

```
#include <iostream>
#include <string>
using namespace std;
```

```
class book {
```

```
    string book - name;
    String author - name;
    string book - price;
```

```
public:
```

```
    void accept () {
        cout << "Enter book name: ";
        getline (cin, this->book - name);
```

```
        cout << "Enter author name: ";
        getline (cin, this->author - name);
```

```
        cout << "Enter book price: ";
        cin >> book - price;
    }
```

```
    void display () {
```

```
        cout << "In Name of book: " << book - name;
```

```
        cout << "In Name of author: " << author - name;
```

```
        cout << "In Book price" << book - price;
```



```

    }
    { b;
int main() {
    book * b, = new book;
    b-> accept();
    b-> display();

    return 0;
}

```

- 2) WAP to declare a class 'student' having data members roll_no and percentage. using 'this' pointer invoke member functions to accept and display this data for one object of the class

```

-> #include <iostream>
    #include <string>
    using namespace std;

```

```

class student {
    int roll_no;
    float percentage;
}

```

```

public
    void accept() {
        cout << "Enter student roll_no:";
        cin >> this->roll_no;
        cout << "Enter student percentage:";
        cin >> this->percentage;
    }
}

```



```
void display () {
```

```
    cout << "In student roll no: " <<
    roll_no;
    cout << "In student percentage: " <<
    percentage;
}
```

```
}
}
```

```
};
```

```
int main () {
```

```
    student s;
```

```
    s.accept();
```

```
    s.display();
```

```
    return 0;
```

```
}
```

3. ~~WAP~~ WAP to demonstrate the use of nested class

→ #include <iostream>
using namespace std;

```
class student {
```

```
public:
```

```
    class marks
```

```
    public:
```

```
    int m1, m2;
```

```
    void accept () {
```

```
        cout << "Enter marks for m1:"
```

```
        cin >> m1;
```

```
        cout << "Enter marks for m2:"
```

```
        cin >> m2;
```



```

void display () {
    cout << "marks m1:" << m1 << endl;
    cout << "marks m2:" << m2 << endl;
}
}
}

```

```

int main () {
    Student s;
    s.accept();
    s.display();
    return 0;
}

```

Q
12/8/25

Experiment - 4

- 1) VAP to swap two numbers from class using objects as function argument write swap function as members function.

→ #include <bits/stdc++.h>

using namespace std;

```
class Number {
    int a, b;
```

public:

```
void Data (int n, int y) {
```

```
    a = x;
```

```
    b = y;
```

```
}
```

```
void display () {
```

```
    cout << "a = " << a << "b = " << b << endl;
```

```
}
```

```
void swap (Number &obj)
```

```
int temp;
```

```
temp = a;
```

```
a = obj.a;
```

```
obj.a = temp;
```

```
temp = b;
```

```
b = obj.b;
```

```
obj.b = temp;
```

```
}
```

```
};
```



```
int main () {
```

```
    Number obj1, obj2;
```

```
    obj1. Data (10, 20);
```

```
    obj2. Data (30, 40);
```

```
    cout << "obj1:";
```

```
    obj1. display();
```

```
    cout << "obj2:";
```

```
    obj2. display();
```

```
    obj1. swap(obj2);
```

```
    cout << "In After swap:" << endl;
```

```
    cout << "obj1:";
```

```
    obj1. display();
```

```
    cout << "obj2:";
```

```
    obj2. display();
```

```
    return 0;
```

```
}
```

out put

obj 1: a = 10 , b = 20
obj 2: a = 30 , b = 40

After swap:

obj 1: a = 30 , b = 40

obj 2: a = 10 , b = 20

2) WAP to swap two numbers from a same class using Friend Function.

→ #include <bits/stdc++.h>
using namespace std;

```
class Number {  
    int value;  
public:  
    void value (int v) {  
        value = v;  
    }  
    void display () {  
        cout << "value: " << value << endl;  
    }  
    friend void swap value (Number & N1,  
        Number & N2)  
};
```

```
void swap values (Number & N1, Number & N2) {  
    int temp = N1.value;  
    N1.value = N2.value;  
    N2.value = temp;  
}
```

```
int main () {  
    Number A, B;  
    A.value (10);  
    B.value (20);
```



```
A. display();
B. display();
```

```
swap values (A, B);
```

```
cout << "New values: " << endl;
A. display();
B. display();
```

```
return 0;
```

* Output:-

```
A = 10
B = 20
New values
A = 20
B = 10
```

```
int main()
{
    int A, B;
    A = 10;
    B = 20;
    swap(A, B);
    cout << "New values: " << endl;
    A. display();
    B. display();
    return 0;
}
```

Q3] WAP to swap two numbers from different class using friend function.

⇒ #include <bits/stdc++.h>
using namespace std;

```
class B;  
class A {  
    int numA;
```

```
public:
```

```
    void value (int a) {  
        numA = a;  
    }
```

```
    void display () {  
        cout << "class A value = " << numA << endl;  
    }
```

```
    friend void swap values (class A, class B);  
};
```

```
class classB {  
    int num B;
```

```
public:
```

```
    void value (int b) {  
        num B = b;  
    }
```

```
    void display () {  
        cout << "class B value = " << num B << endl;  
    }
```

```
    friend void swap values (class A &, class B &);  
};
```



```
void swap values (class A &a, class B &b)
{
    int temp = a.numA;
    a.numA = b.numB;
    b.numB = temp;
}
```

```
int main () {
    class A A;
    class B B;

    A.value (20);
    B.value (30);
    cout << "values: " << endl;
    A.display ();
    B.display ();
    swap values (A, B);
    cout << "swapped values: " << endl;
    A.display ();
    B.display ();
    return 0;
}
```

- 4) WAP to create two classes Result1 & Result2 ~~there~~ which stores the marks. Read values of both classes & object compute the average of two results.

```

#include <bits/stdc++.h>
using namespace std;
class Result2;
class Result1;

float mark1;
public:
    void read marks () {
        cout << "Enter marks for Result 1: ";
        cin >> mark1;
    }

friend float compute Average (Result1 &
Result2 &r2)
{
    class Result2 {
        float marks 2;
        public:
        void read marks () {
            cout << "Enter marks for Result 2: ";
            cin >> mark1;
        }

        friend float computetheir age (Result1 &r1, Re
2 &r2);
    };

float compute average (Result r1, result r2)
{
    return (r1.mark1 + r2.marks2) / 2;
}

```



```

int main() {
    Result1 A1;
    Result2 A2;

```

~~A1.read marks();~~

~~A2.read~~

A1.read marks();

A2.read marks();

```

float avg = computeAverage(A1, A2);
cout << "In Average of two results:"
<< avg << endl;
return 0;

```

output:-

Enter marks for Result1: 10

Enter marks for Result2: 20

Average of two results: 15

5) VAP to find the greatest number among two numbers from different classes using friend function

```

> #include <bits/stdc++.h>
using namespace std;
class classB;
class classA {
    int numA;
public:
    void Accept (int a) {
        numA = a;
    }
    void display () {
        cout << "Class A Number: " << numA << endl;
    }
    friend void find_greatest (classA, classB)
};

class classB {
    int numB;
public:
    void Accept (int b) {
        numB = b;
    }
    void display () {
        cout << "Class B Number: " << numB << endl;
    }
    friend void find_greatest (classA, classB)
};
  
```



```

void find_greatest (class A a, class B b) {
    if (a.numA > b.numB) {
        cout << " greatest number is from class A" << endl;
    }
    else if (b.numB > a.numA) {
        cout << " Greatest number is from class B" << endl;
    }
    else {
        cout << " Both are equal." << endl;
    }
}

```

```

int main () {
    class A a;
    class B b;

```

```

    a.Accept(35);
    b.Accept(40);
    a.display();
    b.display();

```

```

    find_greatest(a, b);
    return 0;
}

```

* output

```

class A Number : 35
class B Number : 40
Greatest Number is from class B : 40

```



```

6) #include <iostream>
using namespace std;

class class A {
private:
    int numA;
public:
    class A(int a) { numA(a); }
    friend int sum (const class A & a, const
    class B & b);
};

class class B {
private:
    int numB;
public:
    class B(int b) { numB(b); }
    friend int sum (const class A & a, const class B & b);
};

int sum (const class A & a, const
class B & b) {
    return a.numA + b.numB;
}

int main()
{
    class A objA(30);
    class B objB(20);
    cout << "Sum = " << sum(objA, objB)
    << endl;
    return 0;
}

```



```

2-> #include <iostream>
using namespace std;

class Number {
private:
    int value;
public:
    Number (int val) { value = val; }
    void show () { cout << "value: " << value << endl; }
    friend void swapNumbers (Number &, Number &);
};

void swapNumbers (Number &n1, Number &n2) {
    int temp = n1.value;
    n1.value = n2.value;
    n2.value = temp;
}

int main () {
    Number num1 (60);
    Number num2 (40);
    cout << "Before swap: " << endl;
    num1.show ();
    num2.show ();

    swapNumbers (num1, num2);
    cout << "After swap: " << endl;
    num1.show ();
    num2.show ();
    return 0;
}

```

```
3->#include <iostream>
using namespace std;
```

```
class Box {
```

```
private:
```

```
int volume;
```

```
public:
```

```
Box(int v): volume(v) {}
```

```
friend void findGreater(const Box&,
const cube&);
```

```
class cube {
```

```
private:
```

```
int volume;
```

```
public:
```

```
cube(int v): volume(v) {}
```

```
friend void findGreater(const Box&, const
cube&);
```

```
void findGreater(const Box& b, const cube
cube& c)
```

```
{
```

```
if (b.volume > c.volume)
```

```
cout << "Box has greater volume" << endl;
```

```
else if (b.volume < c.volume)
```

```
cout << "cube has greater volume"
```

```
endl;
```

```
else
```

```
cout << "Both have equal volume";
```



```
int main() {
    Box box1(20);
    Cube cube1(80);
    findGreater(box1, cube1);
    return 0;
}
```

4

```
#include <iostream>
using namespace std;
```

```
class Complex {
```

```
private:
```

```
int real;
```

```
int imag;
```

```
public:
```

```
Complex(int r=0, int i=0): real(r),
    imag(i) {}
```

```
void show() const {
```

```
    cout << real << " + " << imag << "i" << endl;
}
```

```
Complex add(const Complex& c1, const
    Complex& c2) {
```

```
    return Complex(c1.real + c2.real, c1.imag
        + c2.imag);
}
```

```

int main()
{
    complex a(2, 8);
    complex b(2, 3);
    complex c = add(a, b);
    cout << "Sum of complex numbers: ";
    c.show();
    return 0;
}

```

5

→

```

#include <iostream>
using namespace std;

```

```

class student {
private:
    string name;
    int mark1, mark2, mark3;
public:
    student(string n, int m1, int m2,
            int m3) {
        name(n), mark1(m1), mark2(m2),
        mark3(m3);
    }
}

```

```

friend void calculateAverage(const student& s)
{
}

```

```

void calculateAverage(const student& s) {
}

```



```
float avg = (s.mark1 + s.mark2 + s.mark3) / 3.0f;
```

```
cout << " student name: " << s.name << endl;
cout << " Average marks: " << avg << endl;
```

```
}
```

```
int main ()
{
```

```
    student s1 ("kartik", 90, 98, 92);
```

```
    calculate Average (s1);
```

```
    return 0;
```

```
}
```

6→

```
#include <iostream>
```

```
using namespace std;
```

```
class Alpha {
```

```
private:
```

```
    int a;
```

```
public:
```

```
    Alpha (int val) : a(val) {}
```

```
    friend void totalsum (const Alpha&, const Beta&);
```

```
class Beta {
```

```
private:
```

```
    int b;
```

```
public:
```

```

Beta (int val) : b(val) {}
friend void totalsum (const Alpha&,
const Beta&, const Gamma&);
};

```

```

class Gamma {
private:
int c;

```

```

public:

```

```

Gamma (int val) : (val) {}

```

```

friend void totalsum (const Alpha&,
const Beta&, const Gamma&);
};

```

```

void totalsum (const Alpha& x, const
Beta& y, const Gamma& z);

```

```

{ int sum = x.a + y.b + z.c;

```

```

cout << "sum of all values:" << sum
<< endl; }

```

```

int main ()
{

```

```

Alpha obj1 (50);

```

```

Beta obj2 (30);

```

```

Gamma obj3 (20);

```

```

totalsum (obj1, obj2, obj3);

```

```

return 0;
}

```



```

>1 # include <iostream>
    # include <math>
    using namespace std;

    class point {
    Private:
        int x, y;
    Public:
        point (int xval, int yval) : x(xval), y(yval) {}

        friend double calculateDistance (const point p1, const point p2) {
            double calculateDistance (const point p1, const point p2) {
                int dx = p2.x - p1.x;
                int dy = p2.y - p1.y;
                return sqrt(dx * dx + dy * dy);
            }

            int main () {
                point p1 (3, 4);
                point p2 (7, 1);

                double distance = calculateDistance (p1, p2);

                cout << "Distance between points"
                << distance << endl;
                return 0;
            }

```

```

8)
-> #include <iostream>
using namespace std;

class BankAccount {
private:
    int bal anac balance;
public:
    BankAccount (int b) : balance (b) {}
    friend void auditBalance (const BankAccount & acc, const Audit & a);
};

class Audit {
public:
    void startAudit (const BankAccount & acc, const Audit & a) {
        auditBalance (acc, *this);
    }

    friend void auditBalance (const BankAccount & acc, const Audit & a);
};

void auditBalance (const BankAccount & acc, const Audit & a) {
    cout << "Auditing account ... \n";
    cout << "Balance: " << acc.balance;
}

```


int main() {
 Bank Account myacc(1200000);
 Audit auditor;
 auditor.startAudit(myacc);
 return 0;
 }

Bank Account (int b): balance(b) {
 friend void addBalance(const BankAccount& a) {
 (b += a.getBalance());
 }

class Audit {
 public:
 void startAudit(const BankAccount& a) {
 const & auditBalance(a, this);
 }

friend void auditBalance(const BankAccount& a, Audit* a_ptr) {
 const & audit(a, a_ptr);
 }

void auditBalance(const BankAccount& a, Audit* a_ptr) {
 acc(a_ptr + a_ptr, a_ptr);
 }

const & audit(const BankAccount& a, Audit* a_ptr) {
 const & audit(a_ptr, a_ptr);
 }

Exp 5

i) ~~Parameter Default:~~

#include <iostream>

using namespace std;

class sum {

int n = 10;

int result = 0;

Public:

sum() {

for (int i = 1; i <= n; i++)

result += i;

}

void display() {

cout << "sum of numbers from

1 to " << n << " is " << result << endl;

}

};

int main() {

sum s;

s.display();

return 0;

}

* O/P

Sum of numbers from 1 to 10 is: 55

* Parameterized:

```
#include (iostream)
using namespace std;
class sum {
    int n;
    int sum = 0;
    public:
    sum (int num) {
        n = num;
        for (int i = 0; i <= n; i++) {
            sum = sum + i;
        }
    }

    void display () {
        cout << "sum = " << sum;
    }
};

int main () {
    sum s1 (5);
    s1.display();

    return 0;
}
```

* O/P

sum = 15

* (copy
 #include <iostream>
 using namespace std;

class sum {
 int n;
 int s = 0;

public:

sum() {
 n = 4;
 }

sum(sum & obj) {
 n = obj.n;

int i;

for (i = 1; i <= n; i++) {
 s = s + i;

}

cout << "sum = " << s;

}

}

int main () {

sum s1;

sum s2(s1);

return 0;

}

* O/P

- sum = 10

Q2 WAP to declare class student having name & %. Write a constructor to initialize these data members. Accept & display for one student.

→
* Default:

```
#include <iostream>
using namespace std;
```

```
class student {
    String name;
    float percentage;
```

```
public:
```

```
    student ()
    {
        name = "Kartik";
        percentage = 90.0;
    }
```

```
    void display ()
    {
        cout << "Name & percentage: " << name
        << " " << percentage;
    }
```

```
};
```

```
int main ()
{
```

```
    student s1;
    s1.display ();
    return 0;
}
```

* Parameterized:

```
#include <iostream>
using namespace std;

class student {
    float per;
    string name;

public:
    student (float p, string n)
    {
        per = p;
        name = n;
    }

    void display {
        cout << "Name:" << name << endl;
        cout << "Percentage:" << per << endl;
    }
};

int main ()
{
    student s1 (90, "kartik");
    s1.display ();

    return 0;
}
```


* Copy

```
#include <iostream>
using namespace std;
```

```
class Student {
    string name;
    float percentage;
```

```
public:
```

```
Student ( string n, float p ) {
    name = n;
    percentage = p;
}
```

```
Student (Student & s)
{
```

```
    name = s.name;
    percentage = s.percentage;
}
```

```
void display () {
```

```
cout << "Name: " << name << endl;
```

```
cout << "Name: " << name << endl;
cout << "percentage: " << percentage
<< "%." << endl;
```

```
}
```

```
};
```

```
int main () {
```

```
    string n;
    float p;
```

```
cout << "Enter name: ";  
getline (n, n);
```

```
cout << "Enter percentage: ";  
cin >> p;
```

```
Student s1 (n, p);  
Student s2 (s1);
```

```
cout << "In student data: " << endl;  
s1.display();  
return 0;
```

Pr

16/10

Q3

```
#include <iostream>
#include <string>
using namespace std;
```

```
class college {
    int roll no;
    string name;
    string course;
```

```
public:
```

```
college (int r, string n, String c = "computer science")
```

```
{
```

```
    roll _ no = r ;
    name = n ;
    course = c ;
```

```
}
```

```
void display ()
```

```
{
```

```
    cout << "Roll No:" << roll - no << endl;
    cout << "Name:" << name << endl;
    cout << "Course:" << course << endl;
```

```
}
```

```
};
```

```
int main ()
```

```
{
```

```
    int r1, r2 ;
    string h1, h2 ;
```

```
cout << "enter roll no & name first  
Student: ";  
cin >> r1 >> r2
```

```
College C1 (r1, n1);  
College C2 (r2, n2);
```

```
cout << "Details of student: ";  
s1.display();  
s2.display();  
return 0;  
}
```

Q1 VAP to demonstrate constructor overloading
-> #include <iostream>
using namespace std;

```
class Number {  
    int x, y;  
    public:  
    Number (int a)  
    {  
        x = a;  
        y = 0;  
    }  
    Number (int a, int b)  
    {  
        x = a;  
        y = b;  
    }  
}
```



```
void display () {
    cout << "x=" << x << " y=" << y << endl;
}
};
```

```
int main () {
    Number n1 (5);
    Number n2 (10, 20);
    n1.display ();
    n2.display ();
    return 0;
}
```

Exp - 6

a) WAP to implement multilevel inheritance.

```
→ #include <iostream>
#include <string>
using namespace std;
class person
{
protected:
    int age;
    string name;
}
class student: public person
{
private:
    int roll_no;
public:
    void accept()
    {
        cout << "Enter your name: ";
        cin >> name;
        cout << "Enter your age: ";
        cin >> age;
        cout << "Enter your roll no: ";
        cin >> roll_no;
    }
    void display()
    {
        cout << "Name: " << name;
cout << "Enter your age: ";
        cin >> name;
        cout << "Enter your age: ";
    }
}
```



```
cin >> age;  
cout << "Enter your age: ";  
cin >> roll_no;  
}
```

```
void display ()  
{
```

```
    cout << "Name: \n" << name;  
    cout << "Age: \n" << age;  
    cout << "Roll No: \n" << roll_no;  
}
```

```
}
```

```
int main ()
```

```
{
```

```
    Student s1;  
    s1.accept();  
    s1.display();  
    return 0;
```

```
}
```

b WAP to implement multiple inheritance

```

-> #include <iostream>
using namespace std;
class Academic
{
protected:
    int marks;
};
class sports
{
protected:
    int score;
};
class Result: protected Academic,
               protected sports.
{
    int total_score = 0;
public:
    void accept()
    {
        cout << "Enter the marks of the student: ";
        cin >> marks;
        cout << "Enter the sports Score of the student: ";
        cin >> score;
    }
    void calculate()
    {
        total_score = marks + score;
    }
}
    
```



```
cout << "The marks of the student  
is: << marks << endl;  
cout << "The sports score of the  
student is: " << score << endl;  
cout << "The total score of the  
student is: " << total_score << endl;  
}  
};
```

```
int main()  
{  
    Result r;  
    r.accept();  
    r.calculate();  
    return 0;  
}
```

WAP to implement hierarchical inheritance

```
> #include <iostream>
using namespace std;
class vehical
{
    protected:
        String brand;
        int model;
};
```

```
class car : protected vehical
{
```

```
    protected:
        String type;
};
```

```
class Electric car: protected car
{
```

```
    int battery capacity;
    public:
```

```
    void accept()
    {
```

```
        cout << "Enter the brand of car:";
        cin >> brand;
```

```
        cout << "Enter the model of car:";
        cin >> model;
```

```
        cout << "Enter the type of car:";
        cin >> type;
    }
```



```

void display ()
{
    cout << " \n The brand of the car: <<
        brand << endl;
    cout << " The model of car: " << model
        << endl;
    cout << " The type of car: " << type
        << endl;
}
}
int main ()
{
    Electric car e;
    e.accept ();
    e.display ();
    return 0;
}

```

d) WAP to implement hybrid inheritance

```
→ #include <string>
   #include <iostream>
   using namespace std;
   class person
   {
   public:
       string name;
       int age;

       void get person Details ()
       {
           cout << "Enter name: ";
           cin >> name;
           cout << "Enter age: ";
           cin >> age;
       }

       void show Details ()
       {
           cout << "Name" << Name << "Age: "
           << Age << endl;
       }
   };

   class student : public person
   {
   public:
       string course;
       void get Details ()
       {
           cout << "Enter course: ";
           cin >> course;
       }
   }
```



```

void ShowDetails()
{
    cout << "course: " << course << endl;
}
};
class Employee : public person
{
public:
    void ShowInternDetails()
    {
        cout << "\n -- Intern Details -- \n";
        Student: show person details();
        Show Employee Details();
    }
};
int main()
{
    Intern it;
    it.Student: get person details();
    it.getDetails();
    it.get employee Details();
    it.Show InternDetails();

    return 0;
}

```

Handwritten signature/initials
16/11

Experiment ->

a) VAP using function overloading

```
-> #include <iostream>
using namespace std;
class Area
{
    private:
        Float l, b;
    public:
        void area(Float l)
        {
            Float area;
            area = l * l;
            cout << "Area of square: " << area
                << endl;
        }
        void area(Float l, Float b)
        {
            Float area;
            area = l * b;
            cout << "Area of rectangle: " << area;
        }
};

int main()
{
    Area a1;
    a1.area(8);
    a1.area(9, 12);
    return 0;
}
```


b WAP using function overloading

```
> #include <iostream>
```

```
using namespace std;
```

```
class sum;
```

```
{
```

```
private:
```

```
int a, b, c, d, e, f, g, h, i, j;  
float k, l, m, n, o;
```

```
public:
```

```
void sum (float k, float l, float m, float  
n, float o)  
{
```

```
float sum;
```

```
sum = k + l + m + n + o;
```

```
cout << "sum of 5 float numbers:" <<  
endl sum << endl;
```

```
}
```

```
void sum (int a, int b, int c, int d, int e,  
int f, int g, int h, int i, int j);
```

```
{
```

```
int sum;
```

```
sum = a + b + c + d + e + f + g + h + i + j;
```

```
cout << "sum of 10 integers" << sum  
<< endl;
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
sum s1;
```

```
s1.sum 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 (1.2, 2.3, 3.4, 4.5, 5.6);
```

```
s1.sum (1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
```

```
return 0;
```

```
}
```

C WAP to implement unary operator.

```
-> #include <iostream>
```

```
using namespace std;
```

```
class Number
```

```
{
```

```
private: int x
```

```
public: void accept()
```

```
{
```

```
cout << "Enter a number: ";
```

```
cin >> x;
```

```
void operator -()
```

```
{
```

```
x = -x
```

```
}
```

```
void display()
```

```
{
```

```
cout << "Negated Number: " << x << endl;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

```
Number n1
```

```
n1.accept();
```



```
-n1;  
n1.display();  
return 0;  
}
```

d WAP to implement unary ++ operator.

```
-> #include <iostream>
```

```
using namespace std;
```

```
class Number  
{
```

```
private: int x;
```

```
public: void accept();  
}
```

```
code << "Enter a number: ";  
cin >> x;
```

```
temp = x;  
}
```

```
void operator ++ ()  
{
```

```
x = ++x;
```

```
}
```

```
void reset()  
{
```

```
x = temp;
```

```
}
```

```
void operator ++ (int)
```

```
{
```

```
x = x++;
```

```
}
```

```
void display ()
```

```
{
    cout << "(Pre) The number is: " << x <<
    endl;
}
```

```
void display 2()
```

```
{
    cout << " (Post) The number is : " <<
    x ;
}
}
```

```
int main ()
```

```
{
    number n1;
    n1. accept ();
    ++ n1;
    n1. display ();
    n1. reset ();
    n1 ++;
    n1. display 2 ();
    return 0;
}
```

Pr
16/10

Exp - 8

a- WAP to overload '+' operator. -

```
→ #include <iostream>
#include <string>
using namespace std;
class myString
{
public:
    string str;
    myString operator +(myString + other)
    {
        myString temp;
        temp.str = str + other.str;
        return temp;
    }
    void display()
    {
        cout << str << endl;
    }
};

int main ()
{
    myString s1, s2, s3;
    s1.str = "xy2"
    s2.str = "pqr"
    s3 = s1 + s2
    cout << "Concatenated string: ";
    s3.display();
    return 0;
}
```

b

→

include <iostream>

include <string>

using namespace std;

{
class login {

protected: string name, password;

public:

virtual void accept();

};

cout << "Enter name:";

cin >> name;

cout << "Enter pass:";

cin >> password;

};

};

class Emaillogin: public login {

string email;

public:

void accept() override {

login: accept();

cout << "Enter email:";

cin >> email;

}

void display();

{

cout << "\n --- Email login Details --- \n";

cout << "Name: " << name << "Password "

<< password << "Email: " << email << endl;

}

};


```
Class membership login : Public login
```

```
{  
    String membership ID;
```

```
public :
```

```
    void accept ()  
        override;
```

```
{
```

```
    login : accept ()  
        override
```

```
{
```

```
    login : accept ()
```

```
    cout << "Enter membership ID: ";
```

```
    cin >> membership ID;
```

```
}
```

```
void display ()
```

```
{
```

```
    cout << "\n -- Membership login Details -- \n"
```

```
    cout << "name: " << name << "In password: "
```

```
    << password << "membership ID: " <<
```

```
    membership ID << endl;
```

```
}
```

```
}
```

```
int main ()
```

```
{
```

```
    Email login e;
```

```
    membership login m;
```

```
    e.accept ();
```

```
    m.accept ();
```

```
    e.display ();
```

```
    m.display ();
```

```
    return 0;
```

```
}
```

16/10

Experiment 4

Q1. C++ program to create new file

```
#include <iostream>
#include <fstream>
using namespace std;

int main () {
    fstream new_file;
    new_file.open("new_file.txt", ios::out);
    if (!new_file.is_open()) {
        cout << "file creation failed!" << endl;
    } else {
        cout << "New file created" << endl;
        new_file.close();
    }
    return 0;
}
```


Q2 implement file writing and copying functionality

```
-> #include <iostream>
#include <fstream>
```

```
using namespace std;
```

```
int main() {
    ofstream file("file1.txt", ios::app);
    if (!file) {
        cout << "Error opening file1.txt" <<
            endl;
        return 1;
    }
```

```
    file << "welcome to MIT" << endl;
    file.close();
```

```
    ifstream file1_read("file1.txt");
    ofstream file2("file2.txt");
```

```
    if (!file1_read) {
        cout << "Error opening file1.txt
            for reading" << endl;
        return 1;
    }
```

```
    if (!file2) {
        cout << "Error opening file2.txt
            for writing" << endl;
    }
```

```
return 1;  
}
```

```
char ch;
```

```
while (file1-read.get(ch)) {  
    file2.put(ch);  
}
```

```
cout << "Content copied successfully  
from file1.txt to file2.txt" << endl;
```

```
file1-read.close();
```

```
file2.close();
```

```
return 0;
```

```
}
```


Q3
→

```
#include <iostream>
#include <fstream>
using namespace std;

int main ()
{
    ifstream inputfile ("input.txt");
    if (!inputfile)
    {
        cout << "cannot open input.txt!"
        << endl;
        return 1;
    }

    char ch;
    int spacecount = 0;
    int digitcount = 0;

    while (inputfile.get(ch))
    {
        if (ch == ' ')
            spacecount++;
        else if (isdigit(ch))
            digitcount++;
    }
```

```
input file.close();
```

```
cout << "Total space: " << spacecount << endl;  
cout << "Total digit: " << digitcount << endl;
```

```
return 0;
```

```
}
```

Pen
|||||

Exp 10

Q1 implement template function to sum array elements

→ #include <iostream>
using namespace std;

```
template <class T>  
T my function (T a[], int n) {  
    T sum = 0;  
    for (int i = 0; i < n; i++) {  
        sum = sum + a[i];  
    }  
    return sum;  
}
```

```
int main () {  
    int a1 [3] = {2, 6, 2};  
    float a2 [3] = {1.2, 1.3, 2.4};  
    double a3 [3] = {10.5, 20.5, 30.5};
```

```
    cout << "sum of integer array is:"  
    << my function (a1, 3) << endl;  
    cout << "sum of float array is:"  
    << my function (a2, 3) << endl;  
    cout << "sum of double array is:"  
    << my function (a3, 3) << endl;
```

```
    return 0;  
}
```

92

```
> #include <iostream>
using namespace std;
```

```
template <class T>
T square (T num) {
    return num * num;
}
```

```
template <>
string square (string str)
{
    return str + str;
}
```

```
int main () {
    int i = 5;
    string s = "Hello";
```

```
cout << "Square of integer" <<
i << " = " << square(i) << endl;
cout << "Square of string" <<
s << " = " << square(s) <<
endl;
```

```
return 0;
}
```


Q3

```
⇒ #include <iostream>
using namespace std;
```

```
template < class T >
class calculator {
    T a, b;
```

```
public:
```

```
    calculator (T x, T y) {
```

```
        a = x;
```

```
        b = y;
```

```
    }
```

```
    T add () { return a + b; }
```

```
    T sub () { return a - b; }
```

```
    T mul () { return a * b; }
```

```
    T div () {
```

```
        if (b != 0)
```

```
            return a / b;
```

```
        else {
```

```
            cout << "Error! Division by  
            zero!" << endl;
```

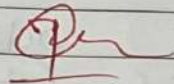
```
            return 0;
```

```
        }
```

```
    }
```

```
};
```

```
int main() {  
    calculator <int> c(10, 5);  
    cout << "Addition = " << c.add() <<  
        endl;  
    cout << "Substraction = " << c.sub() <<  
        endl;  
    cout << "product = " << c.mul() <<  
        endl;  
    cout << "Division = " << c.div() <<  
        endl;  
    return 0;  
}
```


|||||

Q4

```
→ #include <iostream>
using namespace std;
```

```
template <class T>
```

```
class Stack {
```

```
    T stk [10];
```

```
    int top;
```

```
public:
```

```
    Stack () { top = -1; }
```

```
    void push (T val) {
```

```
        if (top == 9)
```

```
            cout << "Stack overflow!"
```

```
            << endl;
```

```
        else
```

```
            stk[++top] = val;
```

```
    }
```

```
    void pop () {
```

```
        if (top == -1)
```

```
            cout << "Stack underflow!" <<
```

```
            endl;
```

```
        else
```

```
            cout << "Popped:" << stk[
```

```
            top-- << endl;
```

```
        }
```

```
void display () {  
    if (top == -1)  
        cout << "Stack is Empty!" << endl;  
    else {  
        cout << "Stack: ";  
        for (int i = top; i >= 0; i--)  
            cout << s[k[i]] << " ";  
        cout << endl;  
    }  
}
```

```
int main () {  
    Stack s (int);  
    s.push (10);  
    s.push (20);  
    s.push (30);  
    s.display ();  
    s.pop ();  
    s.display ();  
}
```

```
return 0;
```

```
}
```


Exp 11

Q1

```
→ #include <iostream>
#include <vector>
using namespace std;

int main () {
    vector<int> v = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

    cout << "Initial vector: " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " << endl;
    }

    cout << "Multiply by 10" <<
    endl;
    for (int i = 0; i < 10; i++) {
        v[i] = v[i] * 10;
    }

    cout << "New vector: " << endl;
    for (int i = 0; i < 10; i++) {
        cout << v[i] << " " <<
        endl;
    }

    return 0;
}
```

```

2) #include <bits/stdc++.h>
    using namespace std;

    vector<int> create() {
        return vector<int>();
    }

    vector<int> modifyelem(vector<int> v/
    {
        int pos, val;
        cout<<"Enter pos of el to be
        modified: ";
        cin>>pos;
        cout<<"Enter value: ";
        cin>>val;
        if (pos <= 0 || pos > v.size()) {
            cout<<"invalid position";
            return v;
        }
        auto it = v.begin() + (pos - 1);
        *it = val;
        return v;
    }

    vector<int> mul(vector<int> v) {
        int n;
        cout<<"Enter number to
        multiply with vector: ";
        cin>>n;
        for (auto it = v.begin(); it !=
            v.end(); ++it) {

```



```

    * it = h;
}
return v;
}

```

```

void disp(vector<int> & v) {
    cout << " ";
    for (auto it = v.begin();
         it != v.end(); ++it) {
        cout << *it << " ";
    }
    cout << " ";
}

```

```

}
int main() {
    vector<int> vec = create();
    vec.push_back(25);
    vec.push_back(35);
    vec.push_back(45);
    vec.push_back(55);

    vec = modifyelem(vec);
    vec = modifyelem(vec);

    vec = mul(vec);
    disp(vec);

    return 0;
}

```

Exp - 12

```
91) #include <bits/stdc++.h>
using namespace std;

int main () {
    stack <int> s;
    s.push (10);
    s.push (20);
    s.push (30);
    s.push (40);

    cout << "Size of stack: " << s.size() <<
    endl;

    s.pop ();

    cout << "Size of stack n: " << s.size() << endl;
    cout << "Element at top of stack: " << s.top() << endl;

    cout << "Stack elements: ";
    while (!s.empty()) {
        cout << s.top() << " ";
        s.pop ();
    }
    cout << endl;
    return 0;
}
```


O/P :

size of stack : 4
 size of stack after pop : 3
 Element at top of stack : 30
 stack elements : 30 20 10

Q2) #include <bits/stdc++.h>
~~#include <queue>~~
 using namespace std;

int main () {

queue <int> q;
 q.push (10);
 q.push (20);
 q.push (30);
 q.push (40);

cout << " size of queue : << q.size();

q.pop();

cout << " In size of queue
 after pop : " << q.size();

cout << " In front element in
 queue : " << q.front();

cout << " In Back element in
 queue : " << q.back();

```
cout << "In queue elements: In";  
while (!q.empty()) {
```

```
    cout << q.front() << " ";  
    q.pop();
```

```
}  
return 0;  
}
```

Q
///